Hardware Interrupts

With an idea as to what functions can do for us, we can move on to a different kind of function—one that is driven by specific hardware in the microcontroller. The reasons for using interrupts are many; maybe we have a lot of code in our `loop()` function and sitting there waiting for a button press would slow things down too much or maybe we might even miss the button press all together. Or instead we might be using a photo sensor or interrupter that triggers when something gets close and it is important to stop a motor right at that exact time. These things are possible with a **hardware interrupt** that can be configured on one of two digital pins to trigger a unique kind of advanced function called an **interrupt service routine** (ISR). When the interrupt is triggered, the ISR will be called regardless of whatever the program is in the middle of. Like using one of the internal timers, monitoring of the hardware interrupt pin happens in the background and the jump to the ISR can happen within four instructions, so fairly quickly. After the ISR has been executed, program flow returns back to where it left off before the interrupt and, if done correctly, with little effect on the rest of the code.

An ISR looks a lot like a regular function but there are a few rules that we need to stick to. First, as was shown in our second project code, the ISR should be kept as short as possible, often only three to four lines of code at the maximum, so as to not overly disrupt the program flow and prevent the interrupt from being triggered again while executing the ISR. If we have a longer ISR than that, we need to disable the interrupt briefly while the ISR is running. Likewise, timing related functions would not work properly within the ISR because they use the same hardware timer. For example, `millis()` will not increment and `delay()` will not work at all. Other than that, an ISR is declared just like a normal function, but in order to use a hardware interrupt we need to first attach it in our code.

attachInterrupt()

The `attachInterrupt()` function enables hardware interrupts and links a hardware pin to an ISR to be called when the interrupt is triggered. This function also specifies the type of state change that will trigger the interrupt. Its syntax follows:

```
attachInterrupt(interrupt, function, mode)
```

The first parameter is the number of the interrupt. On the Arduino Uno there are only two possible hardware interrupts, 0 and 1, which correspond to digital pins 2 and 3 respectively. Note that this parameter refers to the interrupt number and not the pin number. The second parameter is the name of the function that will serve as the interrupt service routine that we will want to execute when the interrupt is triggered. Finally, we have the mode that represents the specific state change that will cause the interrupt to trigger. There are four possible modes, as shown in Figure 7-7, which include `LOW`, `CHANGE`, `RISING`, and `FALLING`.